

[L0-L1 Core Essence & Design Logic]

L0 Essence: xv6-riscv is a teaching-grade RISC-V operating system kernel that implements core UNIXv6 concepts through compact C and assembly, demonstrating privilege isolation (U/S modes), SV39 page tables, trap routing, WAL transaction logging, and pipe synchronization.

L1 Logic:

- Privilege Switch & Trap Routing:** User applications run in U Mode and enter S Mode via `ecall`. The system jumps to the `trampoline` page (mapped at the top of virtual memory in both spaces), saves registers to the `trapframe`, and routes execution to the kernel's `usertrap()`.
- SV39 Page Tables & Memory Isolation:** Each process has a private three-level SV39 page table, while the kernel shares a global mapping. Swapping processes involves updating the `satp` register with the new page directory physical page number and executing `sfence.vma` to flush TLB caches.
- Spinlocks & Sleep/Wakeup Coordination:** Lock synchronization relies on spinlocks (using RISC-V `amoswap` atomic instruction). Process sleep/wakeup coordination uses waiting channels (`sleep` yields the CPU by changing state to `SLEEPING`, and `wakeup` scans the proc table to reschedule them).
- Buffer Cache & WAL Transaction Logging:** The kernel manages a fixed-size block buffer cache (`bcache`) with LRU eviction and enforces crash consistency via Write-Ahead Logging (WAL). Disk metadata updates are written sequentially to a disk journal, and then copied to home sectors after commit.

[L2 OS Data Flow Topology]

- User write call (`write`) → File Descriptor layer (`filewrite`) → Pipe sync layer (`pipewrite`/wakeup reader) OR Inode cache (`writes1`) → Journal transaction layer (`write_log`) → Buffer cache (`bwrite`) → VirtIO Disk Driver (`virtio_disk_rw`) → Hardware Disk Block.

[M1.1] RISC-V Entry & Start Boot ('kernel/entry.S' / 'kernel/start.c')

- Physical Boot Entry:** QEMU loads the kernel and jumps all CPU cores (Harts) to `_entry` in `entry.S`.
- Kernel Stack Setup:** Computes private stack pointers for each CPU using `sp = stack0 + (hartid * 4096)`, allocating a 4KB kernel stack page per core.
- M Mode to S Mode Transition:** In `start.c`, writes the MPP bits in the machine status register `mstatus` to return to Supervisor (S) mode, sets `mepc` to `main()`, configures timer interrupts via `CLINT`, and triggers `mret` to jump to `main()`.

[M1.2] Proc PCB Layout & Allocation ('struct proc' / 'kernel/proc.h' / 'kernel/proc.c')

- Process Control Block:** `struct proc` tracks process state (`UNUSED`, `SLEEPING`, `RUNNABLE`, `RUNNING`, `ZOMBIE`), page table pointer (`pagetable`), kernel stack pointer (`kstack`), trap context (`trapframe`), and open file table.
- Process Allocation:** `allocproc()` searches the global `proc` array, assigns a new PID, calls `kalloc()` to allocate a page for `trapframe`, maps the `TRAMPOLINE` and `TRAPFRAME` pages at the top of the user virtual memory, and configures a dummy kernel context stack frame setting `ra` to `forkret()`.

[M1.3] Round-Robin Scheduler Loop ('scheduler.c' / 'kernel/proc.c')

- Scheduler Thread:** Each CPU runs an infinite `scheduler()` loop that never returns, scanning the global `proc` array for `RUNNABLE` processes.
- Locking Barrier:** Acquires the target process's `p→lock`, changes its state to `RUNNING`, sets the CPU's local `c→proc` pointer, and invokes the context switch.
- CPU Reclaiming:** Calls `swtch(&c→context, &p→context)` to swap registers. When a process yields (`yield()`), sleeps (`sleep()`), or exits (`exit()`), it acquires its own `p→lock` and calls `sched()` to switch back to the scheduler thread.

[M1.4] Context Switch Assembly ('kernel/switch.S' / 'kernel/proc.c')

- switch characteristics:** `swtch` is written in pure RISC-V assembly and only saves callee-saved registers (`ra`, `sp`, `s0 - s11`).
- Execution Flow:**
 - Saves current CPU callee-saved registers (`ra`, `sp`, `s0-s11`) into the `old` context structure.
 - Restores new thread callee-saved registers (`ra`, `sp`, `s0-s11`) from the `new` context structure.
 - Runs `ret` which jumps the instruction pointer to the restored `ra` of the target context.

[M2.1] Privilege Mode & Trap Vector ('stvec' / 'kernel/trap.c')

- Privilege Boundary:** User apps run in U Mode, while the kernel runs in S Mode. Exceptions, system calls, or hardware interrupts automatically raise the privilege level to S Mode.
- Trap Vector Setup:** The kernel writes the virtual address of the assembly routine `uservec` to the `stvec` register before returning to user space.
- Hardware Trap Handling:** When a trap triggers, hardware writes user PC to `sepc`, the trap cause to `scause`, extra info (like page fault address) to `stval`, switches to S Mode, and jumps to `stvec`.

[M2.2] Assembly Trampoline Page ('kernel/trampoline.S')

- Trampoline Page mapping:** The `TRAMPOLINE` page is mapped at the identical high address `0x3FFFFFF00` in both kernel and user page tables. This allows the PC to execute instructions continuously while switching the page table base register `satp`.
- uservec flow:**
 - Uses `carrv` to swap `a0` and `sscratch` (which holds a pointer to the process's user-mapped `trapframe`), releasing a register.
 - Saves all user general registers (`ra`, `sp`, `tp`, `t0-t6`, `a1-a7`, `s0-s11`) into the `trapframe`.
 - Loads kernel page directory `kernel_satp`, kernel stack pointer `kernel_sp`, and `usertrap()` handler address.
 - Writes `satp` with `kernel_satp`, flushes TLB via `sfence.vma`, and jumps to `usertrap()`.

[M2.3] Process Trapframe Structure ('struct trapframe' / 'kernel/proc.h')

- Structure Details:** A dedicated page mapped at `TRAPFRAME` in the user space (just below `TRAMPOLINE`) containing saved registers.
- Core Fields:**
 - `kernel_satp`: Address of the kernel root page table.
 - `kernel_sp`: The process's private kernel stack pointer.
 - `kernel_trap`: Virtual address of the kernel `usertrap` entry point.
 - `epc`: Saved PC when the trap occurred.
 - `regs[32]`: Saved user registers.

[M2.4] Trap Handler & System Call Routing ('usertrap()' / 'kernel/trap.c')

- Trap Router:** `usertrap()` disables interrupts and sets `stvec` to `kernelvec` (to handle interrupts occurring inside kernel space).
- Syscall Dispatching:** If `scause == 8` (System Call):
 - Check if the process is killed, increment `p→trapframe→epc` by 4 (to resume past the `ecall` instruction).
 - Re-enable interrupts to keep the kernel responsive to hardware events.
 - Invoke `syscall()` to execute the system call mapped to `p→trapframe→a7`.
 - Save the return code to `p→trapframe→a0`.
 - Return via `usertrapret()` which jumps back to U Mode.

[M2.5] PLIC & UART Interrupt Dispatching ('kernel/plic.c' / 'kernel/uart.c')

- PLIC Routing:** The Platform-Level Interrupt Controller manages routing of external hardware interrupts (e.g. disk, keyboard input).
- Claim/Complete Loop:**
 - S Mode hardware interrupt triggers `devintr()` (when `scause` indicates an external interrupt).
 - `devintr()` claims the interrupt by reading the PLIC Claim register, which yields the interrupt ID.
 - If ID equals `UART0`, calls `uartintr()` to process characters; if it equals `VIRTIO0`, routes to the disk driver.
 - Writes the processed ID to PLIC Complete register to acknowledge completion.

[M3.1] SV39 Translation & Page Walk ('walk()' / 'kernel/vm.c')

- SV39 layout:** 39-bit Virtual Address is split into L2 index (9 bits), L1 index (9 bits), L0 index (9 bits), and offset (12 bits).
- Page Table Walk Simulation:** `walk(pagetable, va, alloc)` mimics hardware MMU translation:
 - Extracts page directory indices.
 - If the Page Table Entry (PTE) valid bit `PTE_V` is 0, the next-level page directory is missing. If `alloc` is true, calls `kalloc()` to allocate a page, clears it, and writes the base PPN to the PTE.
 - Extracts PPN from PTE, multiplies by 4096 to get the next page table physical base address, and repeats.
 - Returns the pointer to the L0 PTE for `va`.

[M3.2] Kernel Memory Static Mapping ('kvmalloc()' / 'kernel/vm.c')

- Static Layout:** `kvmalloc()` builds the global kernel page table `kernel_pagetable` mapping:
- Hardware registers: Maps `UART0` (`0x10000000`) and `PLIC` (`0x0C000000`) direct-mapped (`VA == PA`).
- Physical memory: Maps RAM starting from `KERNBASE` (`0x80000000`) to `PHYSTOP` (`0x86400000`) with direct mapping (`VA == PA`).
- Trampoline Page: Maps the physical `trampoline` instructions to `TRAMPOLINE` virtual address.

[M3.3] User Address Space & Allocation ('uvm.c' / 'kernel/vm.c')

- User space layout:** Begins at virtual address 0. Page 0 holds code and data, followed by user stack, a Guard Page (mapped invalid, no `PTE_U` flag to prevent stack overflows), and heap pages (grown via `sbrk`).
- Allocation:**
 - `uvmalloc(pagetable, oldsz, newsz)`: Allocates pages via `kalloc()` and maps them using `mappages()` with flags `PTE_U | PTE_W | PTE_R | PTE_V`.
 - `uvmdealloc(pagetable, oldsz, newsz)`: Clears the PTE mappings and frees physical pages.

[M3.4] Copy-on-Write Page Faults ('copy-on-write' Optimization)

- Fork Optimization:** In `fork()`, instead of copying all user memory pages immediately, child and parent page tables are mapped to the identical physical pages.
- Write protection:** Clears the `PTE_W` flag on both page tables and sets a custom flag `PTE_COW`.
- Fault Resolution:** When a write is attempted, the CPU raises a Store Page Fault exception (`scause == 15`) handled in `usertrap()`:
 - Checks if the fault address is a COW page (`PTE_COW` set).
 - Queries page reference counter. If count > 1, calls `kalloc()` to allocate a page, copies contents, maps the page as writable (`PTE_W` set), and decrements the old page reference count.
 - If count == 1, directly sets `PTE_W` and clears `PTE_COW`.
 - Decrements `sepc` by 4 and retries the instruction.

[🔗] OS Kernel Core Trade-offs & Fallacies Matrix

- Eager Copy:** `fork()` should replicate all physical memory pages immediately to preserve process isolation. → **Physical Memory Bandwidth:** Instant deep copying of megabytes of memory blocks creates significant scheduling latency and waste. → **Copy-on-Write (COW):** Share physical pages read-only, allocating and copying only when a write exception occurs.
- Fine-Grained Locking:** Acquire locks on individual PTE entries to maximize concurrent page walks. → **Cache Contention & Deadlock:** Frequent locking and unlocking during high-frequency translation degrades CPU caches and risks deadlocks. → **Coarse Grain Page Table Lock:** Each process manages its own private user page table, utilizing lockless walks and serializing only allocations.
- Synchronous Disk Writes:** Every file `write()` system call must write directly to physical sectors to prevent data loss. → **Disk I/O Latency:** Write times for magnetic/SSD sectors are up to 5 orders of magnitude slower than memory writes. → **Memory Buffer Cache + WAL:** Write updates to memory buffers, write log entries sequentially, commit, and then write asynchronously.
- Recursive Kernel Traps:** Allow kernel exceptions to nest arbitrarily to support complex drivers and page walks. → **Bounded Stack Space:** `xv6` allocates exactly one physical page (4KB) for a process's kernel stack. Deep call graphs cause overflows. → **Strict Non-Nested Traps:** Reset stack pointer `sp` to `trapframe→kernel_sp` on trap entry, disallowing nested exceptions.

[M4.1] Spinlocks & Atomic Instructions ('kernel/spinlock.c' / 'kernel/spinlock.h')

• **Spinlock Acquisition:** `acquire()` performs a loop testing and setting lock state:

```
c
while(__sync_lock_test_and_set(&lk→locked, 1) != 0);
```

This translates to the atomic RISC-V instruction `amoswap.w.aq` which guarantees read-modify-write safety at the hardware level.

• **Interrupt Disabling:** `acquire()` disables local CPU interrupts (`push_off()`) before waiting, preventing a deadlock where a local interrupt handler attempts to acquire the lock held by the interrupted thread.

[M4.2] Sleeplocks & Block Protection ('kernel/sleeplock.c')

• **Disk Lock Protection:** Spinlocks cannot be held while yielding the CPU (which occurs during block I/O). Sleeplocks allow yielding.

• **Sleeplock Structure:** Wraps a spinlock `lk` and a boolean `locked`.

• **Locking operations:**

• `acquiresleep(lk)` acquires the inner spinlock. If `lk→locked` is true, releases the spinlock and sleeps on the channel: `sleep(lk, &lk→lk)`.

• `releasesleep(lk)` sets `lk→locked = 0` and calls `wakeup(lk)` to reschedule threads.

[M4.3] Sleep/Wakeup Synchronization ('sleep()' & 'wakeup()' / 'kernel/proc.c')

• **Condition Variables:** `sleep(chan, lk)` suspends the thread on a pointer value `chan` (wait channel).

• **Lost Wakeup Prevention:** Symmetrically manages lock handover:

1. Acquires `p→lock` (process lock).

2. Releases the resource lock `lk`.

3. Sets state to `SLEEPING` and sets `p→chan = chan`.

4. Calls `sched()` to run the scheduler and yield the CPU.

• **Wakeup routing:** `wakeup(chan)` acquires each process's `p→lock`. If `p→state == SLEEPING` and `p→chan == chan`, sets state to `RUNNABLE`.

[M4.4] Process Termination & Zombies ('exit()' & 'wait()' / 'kernel/proc.c')

• **Resource Cleanup:**

• `exit(status)` closes all open files, releases directory inode references, reparents active children to `initproc` (PID 1), sets state to `ZOMBIE`, wakes up the parent, and yields.

• `wait(addr)` scans the global `proc` table. If a `ZOMBIE` child is found, frees its `trapframe` page, calls `freepagetable()` to release its page table and associated physical frames, reclaims PID, and returns.

[M5.1] Buffer Cache & LRU Eviction ('kernel/bio.c' / 'kernel/buf.h')

• **beache structure:** Tracks 30 memory sector buffers (`struct buf`) in a doubly-linked circular list protected by `beache.lock`.

• **Cache Allocation:**

• **Cache Hit:** `bget()` searches the list for a buffer matching the target `dev` and `blockno`. If found, increments `refcnt` and returns.

• **Cache Miss:** Scans backwards from `beache.head` for a buffer with `refcnt == 0` (LRU candidate). If found, claims it, re-assigns the block number, increments `refcnt`, and returns.

• `brelse(b)` releases reference and moves the buffer to the head of the list, keeping the tail as the least recently used pool.

[M5.2] Write-Ahead Logging (WAL) Model ('kernel/log.c')

• **WAL Principle:** Writes modified metadata pages to a dedicated log area on disk (journal) before committing them to their home directories.

• **Transaction Guard:** `begin_op()` monitors concurrent writes:

• Sleeps if a log commit is underway or if the current transaction plus active transactions exceed the maximum log block capacity.

• Increments `log.outstanding` to merge multiple concurrent system call modifications (e.g. concurrent `create()` calls) into a single disk log transaction.

[M5.3] Log Block Sequential Write ('kernel/log.c')

• **Log writing:** When `end_op()` decrements `log.outstanding` to 0, it initiates the transaction commit phase.

• **Physical serialization:** `write_log()` loops through the dirty block list tracked in memory (`log.1h`):

1. Reads the memory buffer from the `beache`.

2. Writes data sequentially into the disk log block region (starting at sector 2).

3. Updates `log.1h` mapping with the target sector mapping table.

[M5.4] Log Commit & Atomicity ('kernel/log.c')

• **Atomic Boundary:** The write-head call is the critical transition point.

• **Commit sequence:** `write_head()` writes the memory log header structure (`log.1h.n` representing count and block mappings) to log sector 1.

• **Durability guarantee:** If the system crashes before the header sector write completes, the transaction is discarded on reboot. If it crashes after, the transaction is replayed from the log blocks during startup.

[M5.5] Log Installation to Home Blocks ('kernel/log.c')

• **Sector Copying:** After the log header write completes, the transaction is officially committed, and `install_trans()` is invoked.

• **Copy routine:** Reads blocks from the log region, writes them to their true physical sectors on disk (Home Blocks), and updates `beache` statuses.

• **Transaction Completion:** Clears dirty flags on `beache` blocks after the copy completes.

[M5.6] Log Recovery on Reboot ('kernel/log.c')

• **Recovery Hook:** During boot, `initlog()` calls `recover_from_log()`.

• **Replay routine:**

1. Reads the log header sector.

2. If the block count `n > 0`, it indicates a crash occurred after commit but before install finished.

3. Replays the transaction by copying log blocks to home blocks (re-executing `install_trans()`).

4. Writes `n = 0` to log header sector and flushes to disk to prevent redundant recovery.

[M6.1] Inode Layout & In-Memory Cache ('kernel/fs.c' / 'kernel/fs.h')

• **Disk Inode (struct dinode):** Fixed structure containing type, link count, size, 12 direct block pointers, and 1 indirect block pointer (supports up to $12 + 1024 = 1036$ blocks, approx. 512KB).

• **Memory Inode (struct inode):** Extends disk inode with reference count `ref` and a sleeplock.

• **Cache allocation:**

• `iget(dev, inum)` returns the inode matching `inum`. If cached, increments `ref`.

• `iput(ip)` decrements `ref`. If both `ref` and `nlink` (disk link count) reach 0, frees all associated disk blocks and clears the inode on disk.

[M6.2] Directory Lookups & Links ('kernel/fs.c')

• **Directory Layout:** Directories are files containing arrays of `struct dirent` (directories entries mapping `inum` and `name`).

• `dirlookup(dp, name, poff)` reads directory file blocks and searches for a matching name, returning `iget(dp→dev, de.inum)` on hit.

• **Linking:** `dirlink(dp, name, inum)` appends a new `struct dirent` directory record. Creating a hard link increments the target inode `nlink` count and writes a new directory entry.

[M6.3] File Descriptor Table & File abstraction ('struct file' / 'kernel/file.c')

• **Unified Abstraction:**

1. **Process Space:** `p→ofile[fd]` is a process-private array of pointers indexable by file descriptors.

2. **Global Table:** Points to a global file table `ftable` containing `struct file` entries tracking offset `off` and global reference count `ref`.

3. **Underlying File Type:** An open file can wrap an INODE (normal file/device) or a PIPE.

• **Management:** `filealloc()` scans `ftable` to allocate a file entry. `fileclose()` decrements `ref` and dispatches `iput()` or `pipeclose()`.

[M6.4] Pipe Ring Buffers & Synchronization ('kernel/pipe.c')

• **Ring Buffer:** `struct pipe` manages a 512-byte ring buffer (`PIPESIZE`) with pointers `nread` and `nwrite`.

• **Pipe Read/Write Sync:**

• `pipewrite(pipe, addr, n)` writes bytes sequentially. If the buffer fills (`nwrite == nread + PIPESIZE`), calls `wakeup(&pipe→nread)` to awake readers and sleeps on `sleep(&pipe→nwrite, &pipe→lock)`.

• `piperead(pipe, addr, n)` reads bytes. If the buffer empties (`nread == nwrite`), calls `wakeup(&pipe→nwrite)` to awake writers and sleeps on `sleep(&pipe→nread, &pipe→lock)`.

[M6.5] Syscall Parameters & Boundary Check ('kernel/sysfile.c' / 'kernel/syscall.c')

• **Kernel Boundary Protection:** User programs pass pointers via registers which can be arbitrarily configured, requiring strict validation.

• **Extraction & Validation:**

• `argint(n, &val)` and `argaddr(n, &val)` extract integers or address pointers from registers `a0 - a5`.

• `argstr(n, buf, max)` copies a null-terminated string from user space. It invokes `fetchstr(addr, buf, max)` which reads bytes sequentially, verifying the address is strictly less than the process size `p→sz` to prevent memory disclosure.

[貳] Zone T: xv6-riscv Diagnostics & Hardware Control Registers Reference

T1: RISC-V Control Registers & CPU Architecture Reference

sstatus (Supervisor Status Register): Controls supervisor state. The `SIE` bit controls supervisor interrupt enablement. `SPP` records the CPU privilege mode prior to entering `S Mode`. `satp` (Supervisor Address Translation and Promotion Register): Holds physical page number (PPN) of the root page table and configuration bits (`MODE=8` enables `SV39` translation). `sepc` (Supervisor Exception Program Counter): Holds the address of the instruction that caused the trap. Executing `sret` jumps the CPU PC back to `sepc`. `cause` (Supervisor Exception Cause Register): Trap cause code register. The MSB indicates if the trap is an interrupt (1) or exception (0) (e.g. 8 is user syscall, 15 is store page fault, 1 is timer interrupt). `stval` (Supervisor Trap Value Register): Holds trap-specific info (e.g. holds virtual address that caused a page fault, or raw instruction bytes of an invalid opcode). `sie / sip` (Supervisor Interrupt Enable / Pending Registers): `S Mode` interrupt control. The `STIE` bit in `sie` manages timer interrupts, and `SEIE` manages external interrupts.

T2: xv6-riscv Debugging & QEMU / GDB Commands

`\\texttt{\#1}` Run simulation with GDB server enabled (suspended waiting for GDB connection) `\\texttt{qemu-system-riscv64-machine virt-bios none-kernel kernel/kernel-m 128M-smp 3-nographic-drive file=fs.img-directory=.format=raw-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0-S-gdb tcp:26000}\\texttt{\#2}` Launch GDB and connect to QEMU simulation `\\texttt{riscv64-unknown-elf-gdb-ex "target remote localhost:26000"-ex "symbol-file kernel/kernel"}\\texttt{\#3}` View RISC-V CSRs and general registers (inside GDB) `\\texttt{info registers satp sstatus sepc cause stval}\\texttt{\#4}` View kernel address layout translation (inside QEMU console, press `Ctrl+A` then `C`) `\\texttt{info mem}` # Prints active `SV39` page table mappings and page permissions `\\texttt{info registers}` # Prints all QEMU virtual CPU registers `\\texttt{\#5}` Break on usertrap entry and trace syscall numbers `\\texttt{break usertrap}\\texttt{break syscall}\\texttt{print p→pid}\\texttt{print x p→trapframe→a7}` # Prints value of register `a7` (the syscall number) `\\texttt{\#6}` Single step context switches (inside GDB) `\\texttt{layout asm}\\texttt{step}\\texttt{t/x}\\texttt{logx}&p→context}` # Dumps saved context registers in thread `struct`